

Задание 1

1. Модуль: general.py

Этот базовый класс обрабатывает общие атрибуты code и name, используя принцип обобщения.

Листинг 1: Модуль general.py

```
1 class general:
2     def __init__(self, code=0, name=''):
3         # Конструктор позволяет задать начальные значения атрибутов [4]
4         self.setCode(code)
5         self.setName(name)
6
7         # Методы доступа (set/get) обеспечивают инкапсуляцию данных [5]
8     def setCode(self, value): self.__code = value
9     def setName(self, value): self.__name = value
10    def getCode(self): return self.__code
11    def getName(self): return self.__name
```

2. Модуль: department.py

Класс для сущности "Отдел"(department), наследуемый от general.

Листинг 2: Модуль department.py

```
1 from general import general
2
3 class department(general):
4     def __init__(self, code=0, name='', employees=0):
5         # Вызов конструктора предка для инициализации кода и названия [3]
6         general.__init__(self, code, name)
7         self.setEmployees(employees)
8
9     def setEmployees(self, value): self.__employees = value
10    def getEmployees(self): return self.__employees
```

3. Модуль: expense_type.py

Класс для сущности "Вид расходов"(expenseType), включающий описание и лимиты.

Листинг 3: Модуль expense_type.py

```
1 from general import general
2
3 class expenseType(general):
4     def __init__(self, code=0, name='', description='', limit=0.0):
5         # Наследование общих полей от базового класса [3]
6         general.__init__(self, code, name)
7         self.setDescription(description)
8         self.setLimit(limit)
9
```

```

10     def setDescription(self, value): self.__description = value
11     def getDescription(self): return self.__description
12     def setLimit(self, value): self.__limit = value
13     def getLimit(self): return self.__limit

```

4. Модуль: expense.py

Класс "Расход"(expense), который связывает отдел и категорию затрат через агрегацию.

Листинг 4: Модуль expense.py

```

1 from general import general
2
3 class expense(general):
4     def __init__(self, code=0, exp_type=None, dept=None,
5         amount=0.0, date=''):
6         # Инициализация транзакции [4]
7         general.__init__(self, code, name="Транзакция")
8         self.setExpType(exp_type)
9         self.setDept(dept)
10        self.setAmount(amount)
11        self.setDate(date)
12
13    def setExpType(self, value): self.__expType = value
14    def setDept(self, value): self.__dept = value
15    def setAmount(self, value): self.__amount = value
16    def setDate(self, value): self.__date = value
17
18    def getReportStr(self):
19        # Метод для формирования отчетной строки о расходе [9]
20        dept_name = self.__dept.getName() if self.__dept else
21            "Неизвестно"
22        type_name = self.__expType.getName() if self.__expType
23            else "Неизвестно"
24        return f"[{self.__date}] Отдел '{dept_name}' потратил
25            {self.__amount} на '{type_name}'"

```

5. Основная программа: main.py

Сценарий для создания объектов и проверки их взаимодействия.

Листинг 5: Основная программа main.py

```

1 # Создание объектов сущностей (Отдел и Вид расхода) [10, 11]
2 dept1 = department(1, "IT-отдел", 15)
3 type1 = expenseType(101, "Оборудование", "Ноутбуки и периферия",
4     5000.0)
5
6 # Создание конкретной записи о расходе [6]
7 e1 = expense(code=5001, exp_type=type1, dept=dept1,
8     amount=1200.50, date="2023-10-25")
9
10 # Вывод результата работы программы на экран [11]

```

```
9 print(e1.getReportStr())
```

UML Class Diagram (PlantUML)

The relationship reflects that an expense aggregates both a department and an expenseType, while all inherit from general.

```
@startuml
class general {
    -__code
    -__name
    +setCode()
    +getName()
}

class department {
    -__employees
    +setEmployees()
}

class expenseType {
    -__description
    -__limit
    +setLimit()
}

class expense {
    -__expType
    -__dept
    -__amount
    +getReportStr()
}

general <|-- department
general <|-- expenseType
general <|-- expense

expense o-- department
expense o-- expenseType
@enduml
```

Задание 2

Ниже представлено выполнение **Задания 2** для предметной области «Учет внутриофисных расходов» на основе предоставленных источников. В данном задании реализуется механизм сохранения и загрузки данных в формате XML.

Пояснение к выполнению Задания 2

Согласно источникам, реализация системы хранения данных строится на следующих принципах:

1. **Специализированные списки:** Для каждой сущности создаются классы-списки (наследники `generalList`), которые обеспечивают контроль типов и фабричные методы для создания объектов.
2. **Центральный управляющий класс:** Создается класс (в данном случае `expenseTracker`), который объединяет все списки и координирует работу с ними.
3. **Иерархия классов данных:** Базовый класс `data` определяет общие атрибуты для работы с файлами, а его наследник `dataxml` реализует конкретную логику чтения и записи XML через библиотеку `xml.dom.minidom`.

Программная реализация на Python

1. Модуль `deptlist.py` (Список отделов)

Класс для управления коллекцией объектов `department`.

Листинг 6: Модуль `deptlist.py`

```
1 from department import department
2 from generalist import generalList
3
4 class deptList(generalList):
5     def appendItem(self, value):
6         # Проверка типа добавляемого объекта
7         if isinstance(value, department):
8             generalList.appendItem(self, value)
9
10    def createItem(self, code, name, employees=0):
11        # Проверка на уникальность кода перед созданием
12        if code in self.getCodes():
13            print(f'Отдел с кодом {code} уже существует')
14        else:
15            d = department(code, name, employees)
16            self.appendItem(d)
17            return d
```

2. Модуль `typelist.py` (Список видов расходов)

Класс для управления коллекцией объектов `expenseType`.

Листинг 7: Модуль `typelist.py`

```
1 from expense_type import expenseType
2 from generalist import generalList
3
4 class typeList(generalList):
5     def appendItem(self, value):
6         if isinstance(value, expenseType):
7             generalList.appendItem(self, value)
8
```

```

9     def createItem(self, code, name, description='', limit=0.0):
10         if code in self.getCodes():
11             print(f'Вид расходов с кодом {code} уже существует')
12         else:
13             t = expenseType(code, name, description, limit)
14             self.appendItem(t)
15             return t

```

3. Модуль expense_tracker.py (Управляющий класс)

Аналог класса library из источников, объединяющий все данные.

Листинг 8: Модуль expense_tracker.py

```

1 from deptlist import deptList
2 from typelist import typeList
3 from generallist import generallist
4
5 class expenseTracker:
6     def __init__(self):
7         # Инициализация списков для каждой сущности
8         self.__deptList = deptList()
9         self.__typeList = typeList()
10        self.__expList = generallist() # Список для объектов
11        expense
12
13    def clear(self):
14        # Очистка всех данных
15        self.__deptList.clear()
16        self.__typeList.clear()
17        self.__expList.clear()
18
19    # Методы для работы с отделами
20    def createDept(self, code, name, employees=0):
21        return self.__deptList.createItem(code, name, employees)
22
23    def getDept(self, code): return
24        self.__deptList.findByCode(code)
25    def getDeptList(self): return self.__deptList.getItems()
26
27    # Методы для работы с видами расходов
28    def createType(self, code, name, description='', limit=0.0):
29        return self.__typeList.createItem(code, name, description,
30            limit)
31
32    def getType(self, code): return
33        self.__typeList.findByCode(code)
34    def getTypeList(self): return self.__typeList.getItems()
35
36    # Методы для работы с расходами (транзакциями)
37    def createExpense(self, code, exp_type, dept, amount, date):
38        from expense import expense
39        e = expense(code, exp_type, dept, amount, date)
40        self.__expList.appendItem(e)

```

```

37         return e
38
39     def getExpenseList(self): return self.__expList.getItems()

```

4. Модуль dataxml.py (Работа с XML)

Реализация чтения и записи данных в формате XML.

Листинг 9: Модуль dataxml.py

```

1 import xml.dom.minidom
2 from data import data # Базовый класс данных из
3
4 class dataxml(data):
5     def read(self):
6         # Парсинг XML файла
7         dom = xml.dom.minidom.parse(self.getInp())
8         dom.normalize()
9
10        # Обход узлов XML и создание объектов
11        for node in dom.childNodes.childNodes:
12            if node.nodeType == node.ELEMENT_NODE:
13                # Извлечение атрибутов узла
14                attrs = {name: value for name, value in
15                        node.attributes.items()}
16                if node.nodeName == 'department':
17                    self.getLib().createDept(int(attrs['code']),
18                                             attrs['name'], int(attrs.get('employees',
19                                             0)))
20                elif node.nodeName == 'expenseType':
21                    self.getLib().createType(int(attrs['code']),
22                                             attrs['name'], attrs.get('desc', ''),
23                                             float(attrs.get('limit', 0)))
24                elif node.nodeName == 'expense':
25                    # Восстановление связей по кодам
26                    e_type =
27                        self.getLib().getType(int(attrs['type_code']))
28                    dept =
29                        self.getLib().getDept(int(attrs['dept_code']))
30                    self.getLib().createExpense(int(attrs['code']),
31                                                e_type, dept, float(attrs['amount']),
32                                                attrs['date'])
33
34        def write(self):
35            # Создание структуры XML документа
36            dom = xml.dom.minidom.Document()
37            root = dom.createElement("tracker")
38            dom.appendChild(root)
39
40            # Сохранение отделов
41            for d in self.getLib().getDeptList():
42                node = dom.createElement("department")
43                node.setAttribute('code', str(d.getCode()))
44                node.setAttribute('name', d.getName())

```

```

36         node.setAttribute('employees', str(d.getEmployees()))
37         root.appendChild(node)
38
39         # Сохранение транзакций с указанием кодов связанных объектов
40         for e in self.getLib().getExpenseList():
41             node = dom.createElement("expense")
42             node.setAttribute('code', str(e.getCode()))
43             node.setAttribute('amount', str(e.getAmount()))
44             node.setAttribute('date', e.getDate())
45             node.setAttribute('dept_code',
46                               str(e.getDept().getCode()))
47             node.setAttribute('type_code',
48                               str(e.getExpType().getCode()))
49             root.appendChild(node)
50
51         # Запись в файл с красивым форматированием
52         with open(self.getOut(), "w", encoding='utf-8') as f:
53             f.write(dom.toprettyxml())

```

5. Демонстрация работы (main2.py)

Программа для проверки функционала.

Листинг 10: Демонстрация работы main2.py

```

1 from expense_tracker import expenseTracker
2 from dataxml import dataxml
3
4 # Создание экземпляра трекера и объекта данных
5 tracker = expenseTracker()
6
7 # Загрузка данных (если файл существует)
8 # dat = dataxml(tracker, inp='old_expenses.xml',
9                 out='new_expenses.xml')
10 # dat.read()
11
12 # Демонстрационное наполнение данными
13 d1 = tracker.createDept(1, "Бухгалтерия", 5)
14 t1 = tracker.createType(101, "Канцтовары", "Закупка бумаги и ручек", 2000.0)
15 tracker.createExpense(5001, t1, d1, 150.50, "2023-11-20")
16
17 # Вывод отчета в консоль
18 for e in tracker.getExpenseList():
19     print(e.getReportStr())
20
21 # Сохранение в файл
22 dat = dataxml(tracker, out='expenses_report.xml')
23 dat.write()
24
25 print("Данные успешно сохранены в XML.")

```

Диаграмма классов UML

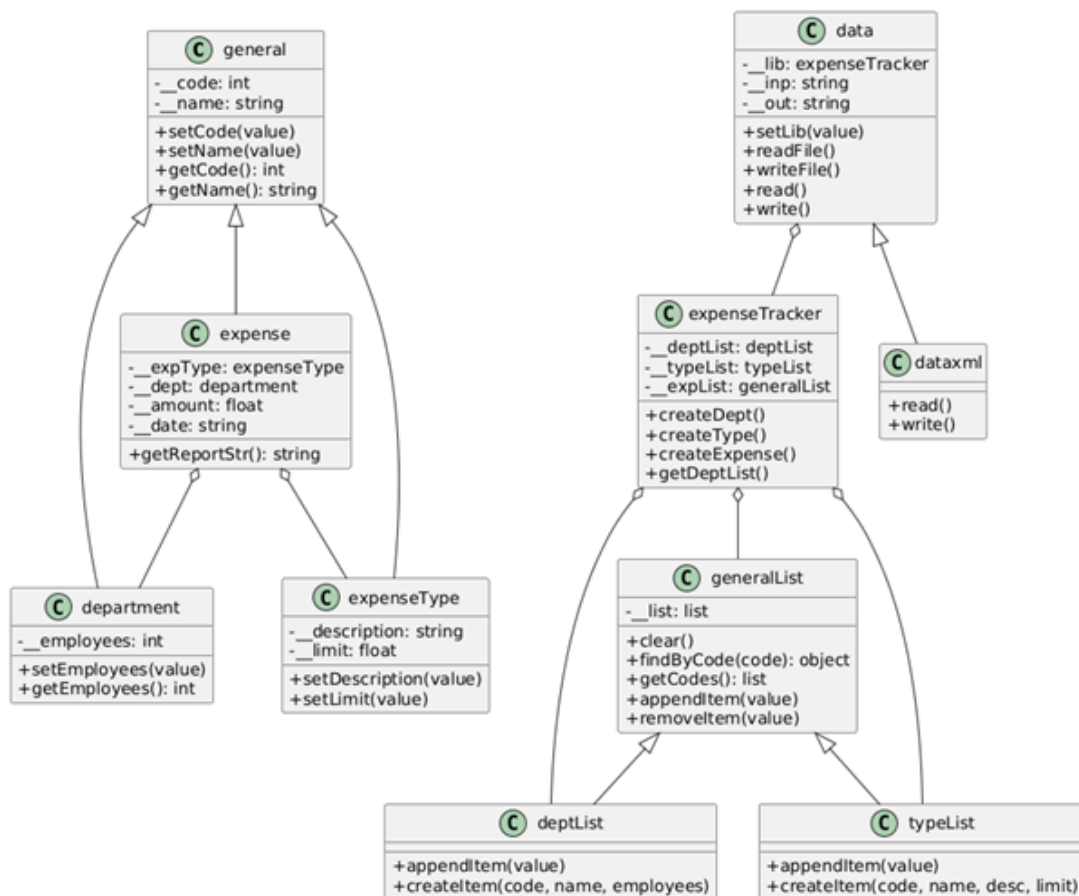


Рис. 1: Диаграмма классов UML

В соответствии с источниками, структура программы выглядит следующим образом:

- `general` является предком для всех сущностей.
- `generalList` является предком для `deptList` и `typeList`.
- `expenseTracker` агрегирует списки сущностей.
- `dataxml` наследует от `data` и взаимодействует с `expenseTracker` для загрузки/сохранения данных.

Задание 3

В соответствии с предоставленными источниками, **Задание 3** заключается в реализации механизмов чтения и записи данных в базу данных **SQLite**. Это логическое продолжение предыдущих заданий, где мы использовали классы сущностей и управляющий класс для работы с внешним хранилищем.

Пояснение к выполнению Задания 3

Для реализации работы с SQLite необходимо:

1. **Описать структуру таблиц:** Создать SQL-скрипт для генерации таблиц, соответствующих нашим сущностям («Отдел», «Вид расходов», «Расход»), с учетом связей (Foreign Keys).
2. **Реализовать класс datasql:** Этот класс наследуется от базового класса data и переопределяет методы read() и write() для работы с библиотекой sqlite3.
3. **Связать данные:** При чтении из базы данных необходимо восстановить связи между объектами (например, найти объект отдела по его коду, сохраненному в таблице расходов).

Программная реализация на Python

1. Модуль datasql.py (Работа с SQLite)

Этот модуль содержит SQL-схему и логику взаимодействия с базой данных.

Листинг 11: Модуль datasql.py

```
1 import sqlite3 as db
2 from data import data # Базовый класс из источника
3
4 # SQL-скрипт для создания пустой базы данных
5 emptydb = """
6 PRAGMA foreign_keys = ON;
7
8 CREATE TABLE department (
9     code INTEGER PRIMARY KEY,
10    name TEXT,
11    employees INTEGER
12 );
13
14 CREATE TABLE expense_type (
15     code INTEGER PRIMARY KEY,
16     name TEXT,
17     description TEXT,
18     limit_val REAL
19 );
20
21 CREATE TABLE expense (
22     code INTEGER PRIMARY KEY,
23     amount REAL,
24     date_str TEXT,
25     dept_code INTEGER REFERENCES department(code) ON UPDATE
26         CASCADE ON DELETE SET NULL,
27     type_code INTEGER REFERENCES expense_type(code) ON UPDATE
28         CASCADE ON DELETE SET NULL
29 );
30 """
31
32 class datasql(data):
33     def read(self):
34         # Установка соединения с БД
35         conn = db.connect(self.getInp())
36         curs = conn.cursor()
```

```

35
36     # 1. Чтение отделов
37     curs.execute('SELECT code, name, employees FROM
38         department')
39     for r in curs.fetchall():
40         self.getLib().createDept(r[0], r[1], r[2])
41
42     # 2. Чтение видов расходов
43     curs.execute('SELECT code, name, description, limit_val
44         FROM expense_type')
45     for r in curs.fetchall():
46         self.getLib().createType(r[0], r[1], r[2], r[3])
47
48     # 3. Чтение транзакций (расходов) и связывание объектов
49     curs.execute('SELECT code, amount, date_str, dept_code,
50         type_code FROM expense')
51     for r in curs.fetchall():
52         dept = self.getLib().getDept(r[3])
53         e_type = self.getLib().getType(r[4])
54         self.getLib().createExpense(r[0], e_type, dept, r[1],
55             r[2])
56
57     conn.close()
58
59 def write(self):
60     # Создание файла БД и таблиц
61     conn = db.connect(self.getOut())
62     curs = conn.cursor()
63     curs.executescript(emptydb)
64
65     # Сохранение данных об отделах
66     for d in self.getLib().getDeptList():
67         curs.execute("INSERT INTO department(code, name,
68             employees) VALUES(?, ?, ?)",
69             (d.getCode(), d.getName(),
70              d.getEmployees()))
71
72     # Сохранение данных о видах расходов
73     for t in self.getLib().getTypeList():
74         curs.execute("INSERT INTO expense_type(code, name,
75             description, limit_val) VALUES(?, ?, ?, ?)",
76             (t.getCode(), t.getName(),
77              t.getDescription(), t.getLimit()))
78
79     # Сохранение расходов с указанием внешних ключей
80     for e in self.getLib().getExpenseList():
81         curs.execute("INSERT INTO expense(code, amount,
82             date_str, dept_code, type_code) VALUES(?, ?, ?, ?,
83             ?)",
84             (e.getCode(), e.getAmount(), e.getDate(),
85              e.getDept().getCode(),
86              e.getExpType().getCode()))

```

```

76         conn.commit()
77         conn.close()

```

2. Демонстрация работы (main3.py)

Программа для проверки корректности переноса данных из XML в SQLite и обратно.

Листинг 12: Демонстрация работы main3.py

```

1 from expense_tracker import expenseTracker
2 from dataxml import dataxml
3 from datasql import datasql
4 import os
5
6 # Инициализация двух менеджеров для проверки синхронизации
7 lib1 = expenseTracker()
8 lib2 = expenseTracker()
9
10 # Объекты для работы с форматами
11 dxml = dataxml(lib1, inp='expenses.xml')
12 dsql = datasql(lib1, out='expenses.sqlite')
13
14 # 1. Загружаем данные из XML, созданного в Задании 2
15 if os.path.exists('expenses.xml'):
16     dxml.read()
17     print("Данные загружены из XML.")
18
19 # 2. Записываем эти данные в SQLite базу
20 if os.path.exists('expenses.sqlite'): os.remove('expenses.sqlite')
21 dsql.write()
22 print("Данные успешно перенесены в SQLite.")
23
24 # 3. Читаем данные из SQLite во второй объект трекера для проверки
25 dsql_read = datasql(lib2, inp='expenses.sqlite')
26 dsql_read.read()
27 print("\nПроверка данных из SQLite:")
28 for e in lib2.getExpenseList():
29     print(e.getReportStr())

```

Диаграмма классов UML

Согласно источникам, в системе добавляется ветка `datasql`, которая, наряду с `dataxml` и `datajson`, расширяет базовый класс `data`.

```

@startuml
skinparam linetype ortho

```

```

class general {
    __code
    __name
}

class expenseTracker {

```

```

        +createDept()
        +createType()
        +createExpense()
    }

class data {
    +readFile()
    +writeFile()
    +read()
    +write()
}

class datasql {
    +read()
    +write()
}

class dataxml {
    +read()
    +write()
}

data <|-- datasql
data <|-- dataxml
data o-- expenseTracker : "управляет данными"

entity "department" as d_table {
    *code : integer
    name : text
    employees : integer
}

entity "expense" as e_table {
    *code : integer
    amount : real
    dept_code : integer <<FK>>
    type_code : integer <<FK>>
}

d_table ||--o{ e_table : "связь 1:N"
@enduml

```

Результат: Теперь система поддерживает полноценное хранение данных в реляционной базе данных SQLite, сохраняя при этом объектно-ориентированную структуру.